

1-4 · Dockerfile 작성 및 이미지 최적화 · 실습 1.5H

이미지 빌드 실습

파이썬 앱을 이미지로 굽고, 캐시가 깨지는 것을 눈으로

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

1.5시간 뒤에는

① 파이썬 앱과 Dockerfile 을 직접 쓰고 docker build 로 이미지를 만든다

② 재빌드 로그에서 CACHED 층을 찾는다

③ 나쁜 순서로 다시 빌드해 재빌드 시간 차이를 time 으로 쟀다

④ .dockerignore 로 빌드 컨텍스트 크기를 줄인다

time 이 찍어 준 real 값은 정리·회고 시간에 서로 공유합니다

여섯 단계 — ~/data-eng/week1/app 한 폴더에서 끝까지

1-3 에서 쓴 실습 폴더 아래에 app 폴더를 새로 만들고 시작합니다

1. 앱 준비	2. Dockerfile	3. 빌드·실행	4. 재빌드	5·6. 실패·정리
app.py requirements.txt	여섯 줄 작성	data-app:0.1	CACHED 찾기	나쁜 순서 .dockerignore

Dockerfile 여섯 줄 — 한 줄씩 뜻

줄	뜻
FROM python:3.12-slim	실행에 필요한 최소 베이스
WORKDIR /app	컨테이너 안 작업 디렉터리
COPY requirements.txt .	의존성 목록 먼저(잘 안 바뀜)
RUN pip install ...	의존성 설치 — 결과가 층에 굳는다
COPY ..	소스는 나중에(자주 바뀜)
CMD ["python", "app.py"]	컨테이너 시작 때 실행

먼저 app.py 와 requirements.txt(requests 한 줄) 두 파일을 같은 폴더에 만든다 — ls 로 확인

--no-cache-dir: pip 이 내려받은 설치 파일을 이미지에 남기지 않아 크기를 줄이는 옵션

핵심 명령 — 빌드 · 확인 · 실행 · 재빌드

빌드	<code>docker build -t data-app:0.1 .</code>
확인	<code>docker images data-app</code>
실행	<code>docker run --rm data-app:0.1</code>
한 줄 추가	<code>echo 'print("..")' >> app.py</code>
재빌드	<code>docker build -t data-app:0.2 .</code>

--rm 은 종료 때 컨테이너를 자동 삭제 · -t 는 이미지 이름:태그 · 마지막 점(.)이 빌드 컨텍스트

빌드하고, 한 줄 고치고, 다시 빌드 — 로그를 읽는다

1

빌드가 오류 없이 끝난다

docker images data-app 에 0.1 태그가 보인다

2

실행하면 두 줄이 찍힌다

「데이터 파이프라인 앱이 실행되었습니다.」와 파이썬 버전 3.12.x

3

app.py 에 한 줄 더하고 0.2 로 재빌드

소스만 바뀌었으니 의존성 층은 그대로 — pip install 단계가 CACHED 로 순식간에 지나간다

빌드 로그에서 찾을 글자

CACHED

COPY ..를 먼저 쓰는 나쁜 순서 — 시간을 재서 비교

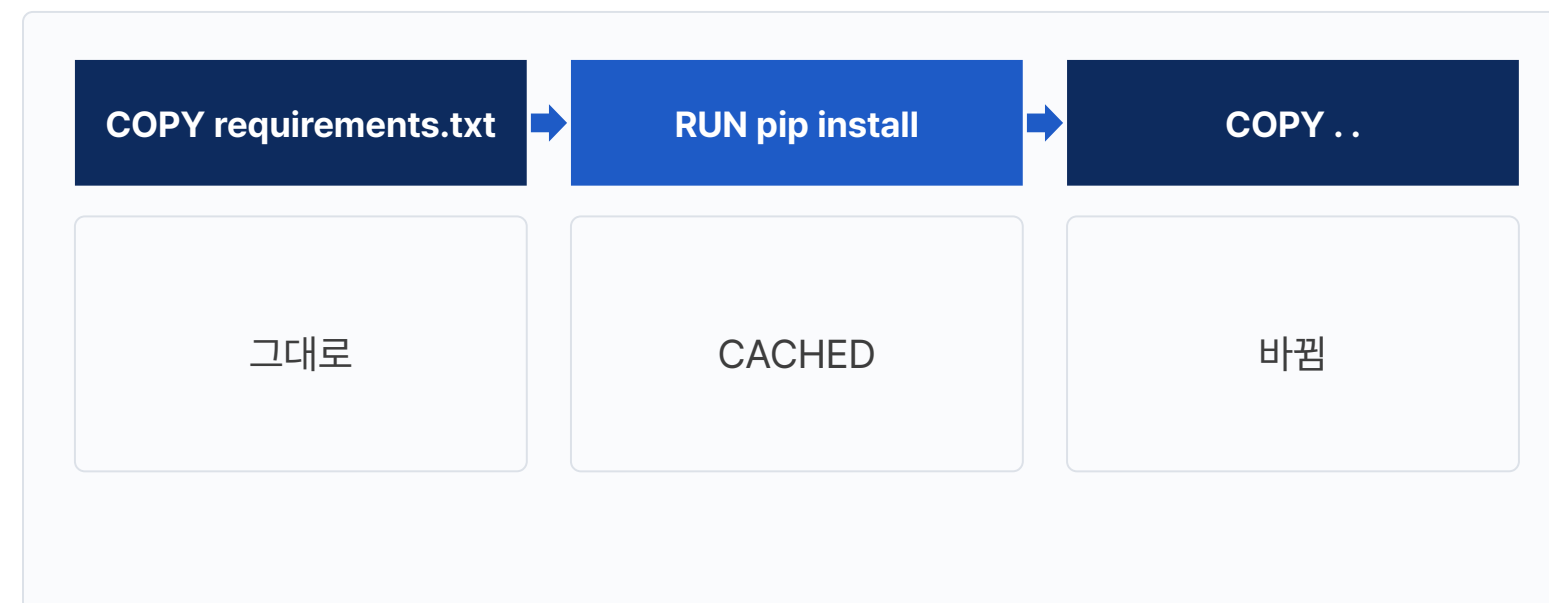
이론에서 본 「순서가 나쁘면 매번 다시 설치한다」를 직접 겪어 봅니다

Dockerfile.bad	처음 빌드	한 줄 수정	time 으로 재빌드	real 값 비교
COPY ..가 pip install 앞	-f Dockerfile.bad 둘 다 오래 걸림	app.py 에 print 한 줄	good2 · bad2 각각	정리·회고에서 공유

같은 한 줄을 고쳤는데 — 몇 초와 몇 분의 차이

Dockerfile — 좋은 순서

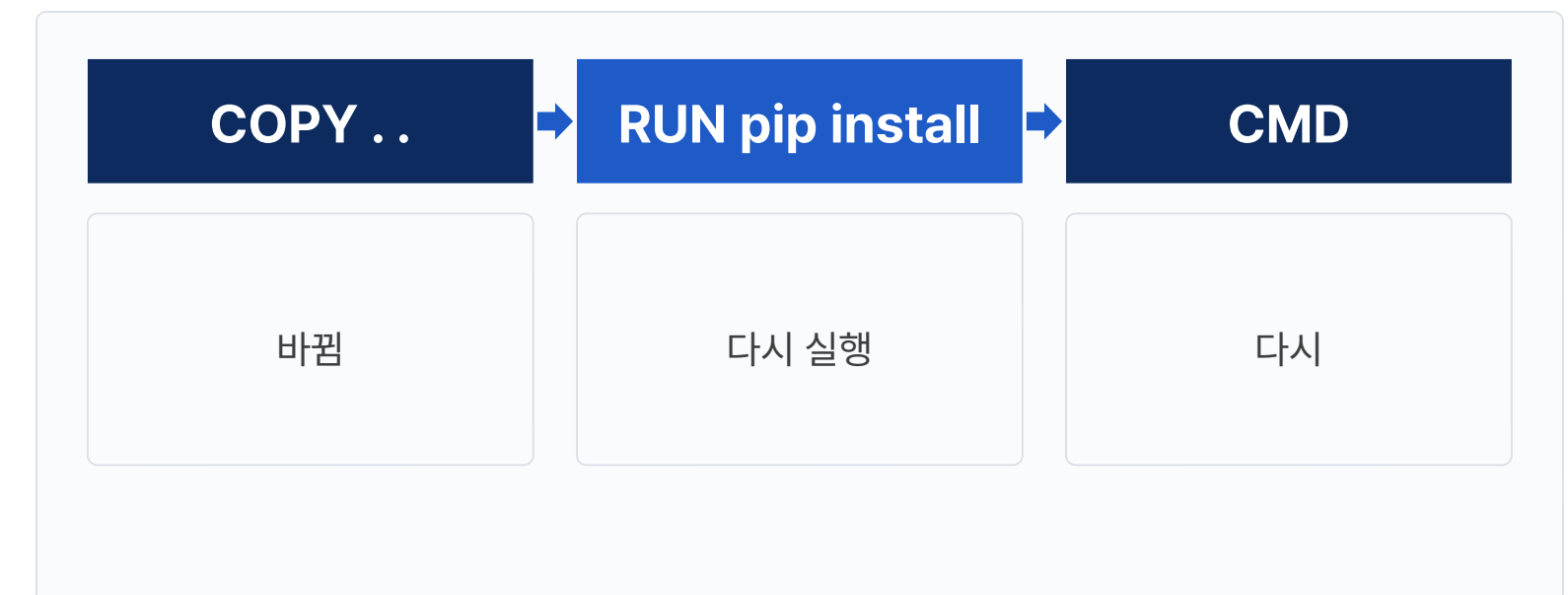
requirements.txt 가 그대로라 pip install 층은 CACHED



VS

Dockerfile.bad — 나쁜 순서

소스가 바뀌면 그 아래층이 전부 깨져 pip install 이 다시 돈다



.dockerignore — 빌드에 필요 없는 것을 전송에서 뺀다

1 Dockerfile.bad · __pycache__ / · *.pyc

이미지에 들어갈 이유가 없는 실험 파일과 파이썬 캐시

2 .git/ · *.log

이력과 로그 — 컨텍스트만 키운다

3 다시 빌드 data-app:0.3

빌드 첫 줄의 transferring context 크기가 이전보다 줄었는지 비교

4 0.3 이 이날의 최종 후보

다음 1H 최적화에서 크기를 더 다듬고 1-5 Compose 의 app 서비스로 쓴다

지금은 패키지가 requests 하나라 몇 초 차이 — pandas·pyspark 등 수십 개면 몇 분, 하루 스무 번 재빌드하면?

이 1.5시간이 끝날 때 확인할 것

- 1 ls 에 app.py·requirements.txt 두 파일이 보인다
- 2 docker run 에 '데이터 파이프라인 앱이 실행되었습니다.'가 찍힌다
- 3 재빌드 로그에서 pip install 단계가 CACHED 다
- 4 나쁜 순서는 pip install 이 다시 돌아 real 값이 훨씬 크다
- 5 .dockerignore 뒤에 transferring context 크기가 줄었다

1-4 · 실습 1.5H 마무리

다음은 이미지 최적화 (1H)

베이스 이미지 교체와 멀티스테이지 빌드로 크기를 줄이고, 전후를 수치로 비교합니다

✓ python:3.12 전체 베이스로 빌드해 크기 비교

✓ RUN 을 합쳐 층수 줄이기

✓ 멀티스테이지 빌드 맛보기

✓ docker history 로 층별 크기 보기

가져갈 것

time 의 real 값 두 개(좋은·나쁜 순서)